

K10: A COMPACT CONVOLUTIONAL NETWORK FOR GLOBAL KEY ESTIMATION

John Hurliman

Independent researcher, Oakland, CA, USA

jhurliman@jhurliman.org

ABSTRACT

K10 is the MIREX 2026 Audio Key Detection submission from the open-source jams project. It is a single 109k-parameter convolutional network that maps a whole-track log-magnitude constant-Q transform to one of 24 major or minor keys in one CPU forward pass, packaged as a script with bundled weights and a locked dependency set. The network was trained only on the curated Beatport-preview annotations of mirdata’s `beatport_key` corpus, with every selection made by cross-validation inside that corpus. On the 567 usable GiantSteps Key excerpts, a single evaluation of the locked model scored 0.832 weighted (symmetric fifth credit) and 0.780 exact, against 0.833 and 0.773 for madmom’s default key processor on the same tracks; the paired difference crosses zero under both fifth-credit conventions, so no ranking between the two is claimed. GiantSteps Key had been used earlier in the project, which is disclosed in full below and was confirmed as admissible by the task captains.

1. SYSTEM

Input features. Audio is decoded, downmixed to mono, and resampled to 22,050 Hz. A constant-Q transform with 24 bins per octave is computed over 8 octaves from C1 with hop 4,096 (about 5.4 frames per second) plus a 4-semitone margin on each side (208 bins), and the margin is cropped after the transform, leaving 192 bins. Magnitudes are compressed with $\log(1 + x)$. The whole track is one input; nothing is cropped or windowed at inference time.

Architecture. Table 1 lists the layers. Three blocks, each two 3×3 convolutions with batch normalization and ELU followed by 2×2 max pooling, reduce the $192 \times T$ input to 64 channels at $24 \times T/8$. After dropout, average pooling over time only leaves a 64×24 map, which is flattened into one linear layer with 24 output logits ordered tonic-major, tonic-minor for the twelve tonics; the prediction is the argmax (no softmax is needed at inference). The frequency positions are kept through to the readout by design: K10’s dense readout uses absolute pitch position, and an earlier variant with global pooling over both axes failed in

layer	output $C \times F \times T$	params
log-CQT input	$1 \times 192 \times T$	
block 1, 16 channels	$16 \times 96 \times T/2$	2,544
block 2, 32 channels	$32 \times 48 \times T/4$	14,016
block 3, 64 channels	$64 \times 24 \times T/8$	55,680
Dropout2d 0.2, mean over time	$64 \times 24 \times 1$	
flatten, linear	24 logits	36,888

Table 1. K10 layers. Each block is two 3×3 convolutions, each followed by batch normalization and ELU, then 2×2 max pooling. Parameter counts include the batch-normalization affine weights; the total is 109,128.

cross-validation. This is an implementation choice for this readout, not a general claim about frequency pooling; the class-channel head of the all-convolutional network in [1], the family K10 belongs to, pools globally. The model has 109,128 trainable parameters (450 kB of weights) and no ensemble.

Training. Table 2 gives the recipe of the run that produced the submitted weights. Augmentation transposes each training example by an integer number of semitones in $[-4, 4]$: the CQT is computed with the 4-semitone margin described above and the 192-bin window is taken at a shifted offset within the padded representation, with the label transposed to match (a shifted crop, not a circular roll). Architecture width and the epoch budget were chosen by 5-fold cross-validation within the training corpus, and the submitted weights are the model retrained on the full corpus at the selected budget. The training script is `eval/train_key_cnn.py` in the repository.

After cross-validation, three pre-registered candidates were compared by cross-validation only, on the 1,129 training tracks shared with the project’s older GiantSteps-MTG feature set, using the out-of-fold CNN posteriors: the standalone CNN (0.749), the existing two-head fusion with the CNN in its second slot (0.747), and the fusion choosing between S-KEY and the CNN (0.749). The standalone CNN was selected as the simplest within noise. The out-of-fold posteriors, the fusion inputs, and the run script are committed with the training records, and `paper/key_candidates_cv.py` recomputes the four numbers from them.



loss / optimizer	cross-entropy over 24 classes; AdamW, learning rate 10^{-3} , weight decay 10^{-4} , cosine annealing over the run
batch / crop	32 examples; one random 320-frame crop (about 59 s) per example per epoch, zero-padded to the longest in the batch
regularization	Dropout2d $p=0.2$ before the pooling; ± 4 -semitone shift augmentation, uniform per example
folds	5; seed-0 shuffle of the sorted mirdata ids dealt round-robin (273/273/273/272/272); not artist-grouped or stratified
selection metric	symmetric-fifth weighted score on the held-out fold, full uncropped tracks, every epoch
budget / stopping	at most 60 epochs; stop 10 epochs after the fold's best
fold results	0.734 (best epoch 53), 0.699 (23), 0.696 (15), 0.731 (17), 0.723 (36); mean 0.716; curves in Fig. 1
final model	all 1,363 tracks for 23 epochs (the median best epoch), cosine schedule over those 23
rejected variant	width 2.0 with patience 15: fold mean 0.710, budget 26
invocation	<code>train_key_cnn.py train</code> <code>-workers 12</code> , every other argument at its default; one A10 GPU, about 2 h

Table 2. Training recipe of the submitted run. All values are from the committed script and the run’s own records (`paper/evidence/key/k10_train/`: fold summaries and per-epoch histories, the training log, the run scripts). The histories confirm the default cap and patience: four folds stopped 10 epochs after their best, and fold 0 reached the 60-epoch cap with its best at epoch 53, so that fold’s best epoch is censored (the median budget is unaffected). An earlier run of the same script with global pooling over both axes reached a chance-level fold mean of 0.159 and was discarded before this run; its log is in the same directory. The committed script differs from the binary that ran only by a `width` field later added to its summary output.

2. DATA AND DATASET-USAGE DISCLOSURE

Training corpus. K10 was trained only on mirdata `beatport_key` [2]: 1,486 Beatport preview excerpts whose key labels are the curated annotations of the GiantSteps-MTG-Keys collection (revised by the dataset authors, not Beatport’s automatic metadata). A track is retained when its first listed annotation is exactly a tonic and *major* or *minor* (flats mapped to sharps); the annotation confidence field is not used. This keeps 1,363 tracks: 1,275 single-key annotations and 88 two-key annotations (for example “E minor | E major”), for which the first listed key is the label. The 123 dropped tracks are the 73 atonal (“X”), “other”, and modally qualified labels (phrygian, dorian, mixolydian, and similar) plus 8 two-key annotations whose first key is of that kind. Audio was present for every track and none was dropped for decoding. The retained set, its folds, and the drop list are reconstructed from the public annotations and cross-checked against the run’s out-of-fold record by `paper/key_train_manifest.py`. The corpus has

zero track-identifier overlap with GiantSteps Key [3]; the check (`eval/verify_key_disjoint.py`) compares Beatport identifiers from the two mirdata indexes (1,486 versus 600, empty intersection). Identifier disjointness does not rule out the same recording appearing under two identifiers. No Isophonics or other key data was used.

Evaluation subset. The development evaluation uses the 567 GiantSteps Key excerpts (of 604 annotated; 600 with retrievable audio) whose public annotation the acquisition script could reduce to one major or minor reference. Many of these are not plain single-key labels: the acquisition script (`eval/acquire_dataset.py`) maps flats to sharps and reduces each label to one major or minor key by mode family (aeolian, dorian, phrygian, and “harmonic” to minor; ionian, mixolydian, and lydian to major). Two-key annotations such as “C major | C minor phrygian” are collapsed by the same rule applied to the whole string: the tonic of the first key, and minor if any minor-family word appears. Of the 567 references, 378 are plain major/minor (including ionian and aeolian), 98 are single keys with another modal qualifier, and 91 are collapsed two-key annotations, of which 15 match neither listed key (“C major | A minor” became C minor). The 33 excluded excerpts carry “other”, “X”, or malformed labels. Both systems were scored against the same references, so the paired comparison is unaffected; the effect on the absolute scores is shown in Section 3.

Prior use of GiantSteps Key in this project. The MIREX 2026 rules ask participants to account for any development use of the evaluation data. The account below is taken from the project’s public experiment ledger (`paper/EXPERIMENTS.md` in the repository), which records every evaluation with its date and commit.

- Before K10 existed, the project evaluated and error-analysed several key systems on the same 567 excerpts: an Essentia template matcher [4, 5], S-KEY [6], madmom’s key CNN [1, 7], and two small logistic refinements of the template output fit on GiantSteps-MTG-Keys.
- One retired predecessor, a mode classifier, had been fit on GiantSteps Key itself. It was identified as contaminated and replaced before any comparison was reported.
- The error analysis of those banked predictions motivated training a stronger base model. That decision is the extent of the benchmark’s influence on K10: its architecture, hyperparameters, and weights were selected by cross-validation within `beatport_key`, under a pre-registered decision rule recorded in the ledger before training.
- The locked K10 model was evaluated exactly once on the 567 excerpts. The descriptive breakdowns in Section 3 were computed afterwards from that one set of predictions; no training or tuning followed. The submitted weights are byte-identical to the evaluated checkpoint
(MD5 6141daf25376c16a7bc4326b742e4a3c).

	sym. fifth	asc. fifth	exact
K10	0.8321	0.8145	0.7795
madmom	0.8328	0.8134	0.7725
K10 — madmom	−0.0007	+0.0011	+0.0071
95% interval	[−0.020, 0.018]	[−0.021, 0.022]	[−0.019, 0.032]
win / loss / tie	41 / 40 / 486	38 / 39 / 490	30 / 26 / 511

Table 3. GiantSteps Key, 567 usable excerpts, one evaluation of the locked K10 model; madmom is its default CNNKeyRecognitionProcessor on the same tracks. Weighted scores use symmetric (sym.) or ascending-only (asc.) fifth credit. K10’s own symmetric-fifth interval is [0.804, 0.858], madmom’s [0.806, 0.859]. All three paired intervals include zero.

	exact	fifth	relative	parallel	other
K10	442	36	24	23	42
madmom	438	35	39	25	30

Table 4. Predictions by symmetric-fifth credit category (counts of 567). K10 makes fewer relative-key confusions and more uncredited errors than madmom; this is descriptive and the totals are not significantly different.

- This history was sent to the task captains before submission; they confirmed on 28 September 2026 that K10 is admissible for the ranked results with this disclosure.

Published GiantSteps Key scores (for example madmom’s 0.746 in [1]) were obtained on different track sets and model configurations and are not matched comparisons with the numbers below.

3. DEVELOPMENT-TIME RESULTS

Table 3 reports the single evaluation, scored two ways. The project’s scorer awards 0.5 for a same-mode key a perfect fifth away in *either* direction, as in [1] and previous MIREX editions; *mir_eval* 0.8.2 [8] awards it only for the ascending fifth. The organizers will report both, so both are given; exact, relative (0.3), and parallel (0.2) credits are identical under the two. Intervals are 95% percentile bootstraps over tracks (10,000 resamples, seed 0). Every number in the table, and in Table 4, is recomputed by `paper/key_scores.py` from the committed per-track predictions and reference labels (`paper/evidence/key/gskey_predictions.jsonl`). Running the submitted package on the same 567 audio files reproduces all 567 committed K10 predictions (`paper/key_rerun_check.py`).

Reference-label sensitivity. Rescoring the same predictions on the 476 references that come from a single annotated key gives K10 0.867 and madmom 0.869 (symmetric fifth; exact 0.826 for both; paired −0.002 [−0.022, +0.018]), and on the 378 plain major/minor references 0.870 and 0.877 (paired −0.007 [−0.029, +0.015]). The collapsed two-key and modal references lower both systems’ absolute scores by about 0.035 and leave the comparison unchanged.

host	wall	peak RSS	install
M2 Max, macOS 26.5	1.6 s	525 MB	cached
i9-14900K, Linux/WSL2	1.7 s	900 MB	46 s

Table 5. Packaged `predict_key.py` on a 120-s MIREX-format WAV, end to end (median of 5 runs on the Mac, 3 on Linux). Install is the one-time dependency download and environment build with a warm network.

These are development-time results on a subset the project had used before, so they give an expectation rather than a prediction of the MIREX score, which is computed on the full 604-excerpt set with the organizers’ scorer and references. No listening audit was performed, so the errors have not been related to musical causes; per-genre means (with counts) are in the output of `paper/key_scores.py` and are descriptive only.

4. SUBMISSION PACKAGE AND MEASUREMENTS

The package contains `predict_key.py` (calling `contract predict_key.py %input %output`; one tab-separated line, tonic then mode, sharps-only tonics), whose PEP 723 inline metadata declares `torch` 2.8, `librosa` 0.10 or later below 1.0, `numpy` 1.26 to 2.2, and `Python` 3.10–3.12; the lockfile `predict_key.py.lock` that fixes the tested environment (`torch` 2.8.0, `librosa` 0.11.0, `numpy` 2.2.6, `soundfile` 0.14.0; `uv run -locked` installs exactly that set); the weights `key_cnn_v1.pt`; a README; and this abstract. Inference uses the same CQT configuration as training. The training pipeline cached features as float16 before casting back to float32, whereas the package keeps float32; on the 567 development excerpts the two paths agree on every argmax, with a maximum absolute softmax difference of 7.5×10^{-5} .

Table 5 gives end-to-end measurements of the packaged command on one 120-second 44.1 kHz 16-bit mono WAV, including interpreter start and model load, with `torch` limited to one thread. The Linux install resolved the default CUDA build of `torch`, which raises install size and resident memory; the GPU is never used. Memory scales with track length through the full-track CQT.

Code and weights are MIT-licensed; the madmom weights used only for the comparison in Table 3 are CC BY-NC-SA and are not part of the submission. The training script, disjointness check, experiment ledger (with the pre-registration and the single-shot result), per-track predictions, and the recompute scripts are in the public repository at <https://github.com/jhurliman/jams>.

5. ACKNOWLEDGMENTS AND AI USE

Thanks to the MIREX organizers for clarifying the eligibility question in advance. Claude (Anthropic) assisted with implementation, experiment execution, and drafting under the author’s direction; OpenAI Codex reviewed the accompanying paper’s code and claims. The author is responsible for the submission.

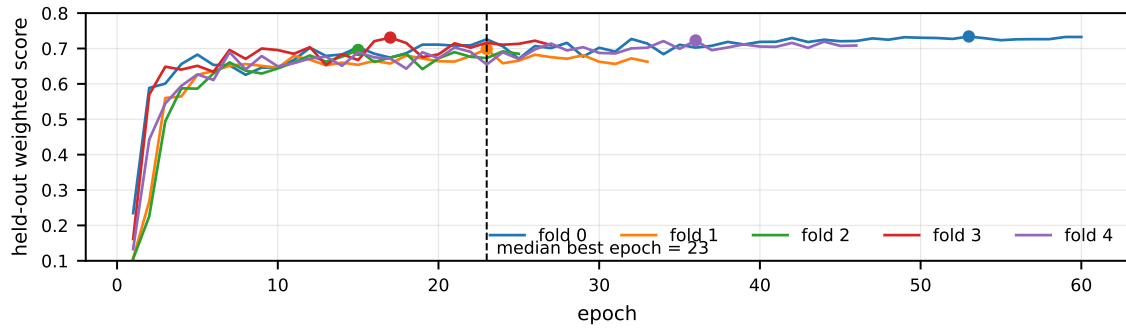


Figure 1. Held-out symmetric-fifth weighted score per epoch for the five cross-validation folds of the submitted run, from the run’s own per-epoch histories (paper/evidence/key/k10_train/). Markers are each fold’s best epoch; the dashed line is their median, the budget used for the final model.

6. REFERENCES

- [1] F. Korzeniowski and G. Widmer, “Genre-agnostic key classification with convolutional neural networks,” in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, Paris, France, 2018.
- [2] Á. Faraldo, “Tonality estimation in electronic dance music: A computational and musically informed examination,” Ph.D. dissertation, Universitat Pompeu Fabra, 2017, the GiantSteps-MTG-Keys dataset accompanies this work.
- [3] P. Knees, Á. Faraldo, P. Herrera, R. Vogl, S. Böck, F. Hörschläger, and M. Le Goff, “Two data sets for tempo estimation and key detection in electronic dance music annotated from user corrections,” in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR)*, Málaga, Spain, 2015.
- [4] Á. Faraldo, E. Gómez, S. Jordà, and P. Herrera, “Key estimation in electronic dance music,” in *Proceedings of the 38th European Conference on Information Retrieval (ECIR)*, Padua, Italy, 2016.
- [5] D. Bogdanov, N. Wack, E. Gómez, S. Gulati, P. Herrera, O. Mayor, G. Roma, J. Salamon, J. R. Zapata, and X. Serra, “Essentia: An audio analysis library for music information retrieval,” in *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)*, Curitiba, Brazil, 2013.
- [6] Y. Kong, G. Meseguer-Brocal, V. Lostanlen, M. Lagrange, and R. Hennequin, “S-KEY: Self-supervised learning of major and minor keys from audio,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.
- [7] S. Böck, F. Korzeniowski, J. Schlüter, F. Krebs, and G. Widmer, “madmom: A new python audio and music signal processing library,” in *Proceedings of the 24th ACM International Conference on Multimedia*, Amsterdam, The Netherlands, 2016.
- [8] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis, “mir_eval: A transparent implementation of common MIR metrics,” in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, Taipei, Taiwan, 2014.